

# 《SSA-based Compiler Design》v–xiii 页对照翻译与解读

范围说明：所给 PDF 实际包含 8 个文件页，对应书页 v–vii ( Foreword ) 与 ix–xiii ( Preface ) ；书页 viii 未包含在文件中。

译法约定：Static Single Assignment form 译作“静态单赋值形式”，简称 SSA ；representation 依语境译作“表示”或“表示形式”；transformation 译作“变换”；analysis 译作“分析”。

## Foreword | 前言

### 书页 v

#### v-1

##### Original

Static Single Assignment (SSA) form is now the dominant framework upon which program analysis and transformation tools are currently built. It is possible to produce a state-of-the-art optimizing compiler that transforms the incoming program into SSA form after the initial syntactic and semantic analysis, and maintain that representation throughout the entire optimization process only to leave SSA form when the final machine code is emitted. This book describes the state-of-the-art representations of SSA form and the algorithms that use it.

##### 译文

静态单赋值 ( Static Single Assignment , SSA ) 形式如今已成为构建程序分析与程序变换工具的主导框架。我们完全可以打造一款最先进的优化编译器：在初步的语法分析和语义分析之后，把输入程序转换成 SSA 形式；在整个优化过程中始终维持这种表示；直到最终生成机器代码时才离开 SSA 形式。本书介绍 SSA 形式的前沿表示方法，以及使用这些表示的算法。

#### v-2

##### Original

The landscape of techniques for program analysis and transformation was very different when the two of us started the initial development of SSA form. Many of the compilers being built at the time were very simple, similar to what a student might generate in a first compiler class. They produced code based on a template for each production in the grammar of the language. The initial optimizing compilers transformed the code based on the branch-free context of the surrounding code, perhaps removing computation if the results were not to be used, perhaps computing a result in a simpler way by using a previous result.

##### 译文

当我们二人最初着手开发 SSA 形式时，程序分析与程序变换技术的面貌与今天大不相同。当时构建的许多编译器都非常简单，类似于学生在第一门编译原理课程中可能写出的编译器。它们针对程序设计语言语法中的每条产生式，套用相应模板来生成代码。早期的优化编译器只根据周围代码中不含分支的局部上下文来变换代码：如果某项计算的结果不会被使用，就可能删除该计算；或者利用先前算出的结果，以更简单的方式得到当前结果。

## Original

The first attempts at more ambitious optimization occurred at IBM, in a few small companies in the Boston area and in a small number of academic institutions. These people demonstrated that large gains in performance were available if the scope of the optimizations was increased beyond the local level. In general, an optimizing compiler can make transformations to an intermediate representation of a program provided that the preconditions exist for the transformation to preserve the semantics of the program. Before SSA form, the dominant provers would construct a summary of what must be true about the state of the program before and after every statement in the program. The proof constructed would proceed by understanding what parts of the state were modified by the statement. This entire process was known as dataflow analysis. The results of that analysis were then used to make transformations to the intermediate code.

## 译文

最早尝试更具雄心的优化工作的，有 IBM、波士顿地区的几家小公司，以及少数几所学术机构。这些先行者证明：只要把优化范围从局部层次扩展开来，就能获得巨大的性能提升。一般而言，只要满足一定的前置条件，使变换能够保持程序语义不变，优化编译器就可以对程序的中间表示实施变换。在 SSA 出现之前，主流的证明方法会为程序中的每一条语句建立摘要，说明该语句执行前后程序状态中必然成立的事实。建立证明时，需要弄清这条语句修改了状态的哪些部分。整个过程称为数据流分析 ( dataflow analysis )。分析所得结果随后用于变换中间代码。

## 书页 vi

## vi-1

## Original

This framework had several drawbacks:

- The represented state was not sparse. The analysis to prove the set of facts involved solving a series of simultaneous equations which proved the validity of every proposition needed for that phase, at every location in the program. Because the state of the entire program had to be represented at each point in the program, the analysis was slow.
- The transformations voided the validity of the analysis. While incrementally updating the proofs was, in theory, possible, such algorithms were impractical because the representations were not sparse. Thus, each pass of the compiler consisted of an analysis phase followed by a transformation phase.
- Each type of transformation had its own set predicates that it needed to prove. Since the analysis was discarded after each transformation, there was no reason to try to prove any fact that was not necessary for the subsequent transformation. This had the effect of making it more difficult to design a new transformation because the programmer had to first figure out exactly what information was needed and then figure out how to perform the proofs before performing the transformation.
- The transformations were fragile. The intermediate representations available in early compilers used the programmer's variable names to express the data dependencies between different parts of the program. Part of designing any transformation involved separating those names into the values that reached each statement.

## 译文

这一框架有几个缺点：

- 所表示的状态不是稀疏的。为了证明一组事实，分析过程必须求解一系列联立方程，从而在程序的每个位置，证明该阶段所需的每个命题都成立。由于程序中的每一点都必须表示整个程序的状态，分析速度很慢。
- 程序变换会使已有分析结果失效。理论上可以增量更新证明，但由于表示并不稀疏，这类算法在实践中不可行。因此，编译器的每一趟处理都由一个分析阶段和紧随其后的一个变换阶段组成。

- 每类变换都有自己需要证明的一组谓词。由于每次变换之后分析结果都会被丢弃，所以没有必要去证明下一次变换用不到的事实。结果是，新变换更难设计：程序员必须先精确判断需要哪些信息，再想清楚如何完成相应证明，最后才能实施变换。
- 变换很脆弱。早期编译器的中间表示使用程序员写下的变量名，来表达程序不同部分之间的数据依赖。设计任何变换时，都要先把这些名字区分成分别能够到达各条语句的具体值。

## vi-2

### Original

While significant work by many people had been done to improve the efficiency of the analysis process, the cost of having to perform the analysis from scratch, coupled with the need to represent every fact at each point, was significant. Fundamentally, this process did not scale: at the time that we developed SSA form, programs were getting larger and there was a desire to make optimizing compilers much more aggressive.

### 译文

尽管许多人已做了大量工作来提高分析过程的效率，但每次都要从头进行分析，再加上必须在每个程序点表示所有事实，代价仍然十分可观。归根结底，这种过程缺乏可扩展性：我们开发 SSA 形式时，程序规模正在不断增大，人们也希望优化编译器能够采用激进得多的优化策略。

## vi-3

### Original

Many people were aiming to fix these problems separately. Def-use chains joined the origin of a value and its use but attempted to do this independently of control flow. Shapiro and Saint refined def-use chains by describing the birthpoints of values. These birthpoints would become the locations where SSA form introduces phi-functions. John Reif used birthpoints for some interesting analyses and found better algorithms to determine them than Shapiro and Saint. But birthpoints were expressed as an analysis technique of the original program and transformations still required rerunning the analysis between phases.

### 译文

许多人试图分别解决这些问题。定义-使用链 ( def-use chain ) 把一个值的来源与它的使用连接起来，但这种做法试图独立于控制流来完成连接。Shapiro 和 Saint 又对定义-使用链作了改进，引入了值的“出生点” ( birthpoint ) 概念。这些出生点后来正是 SSA 形式插入  $\phi$  函数的位置。John Reif 使用出生点完成了一些有趣的分析，并找到了比 Shapiro 和 Saint 更好的出生点判定算法。然而，出生点仍然只是对原始程序实施的一种分析技术；在不同变换阶段之间，仍需重新运行分析。

## vi-4 ( 续至 vii )

### Original

Our initial motivation for SSA form was to build a sparse representation that avoided the cost of proving every fact at every location. Our primary idea was to encode the data and control dependencies into a naming scheme for the variables. We chose to solve the extended form of the constant propagation problem which had been well studied by Wegbreit. We presented the algorithm at the 12th POPL conference. The algorithm was much more efficient than Wegbreit's version, but it also had one very interesting and unique property that was not appreciated at the time: the transformation did not adversely affect the correctness of the representation.

### 译文

我们最初开发 SSA 形式，是为了构造一种稀疏表示，避免在每个位置证明每个事实所带来的代价。我们的核心想法，是把数据依赖和控制依赖编码到变量的命名方案中。我们选择求解常量传播问题的扩展形式；Wegbreit 已对这个问题做过深入研究。我们在第 12 届 POPL 会议上发表了该算法。它比 Wegbreit 的版本高效得多，而且还有一个当时没有得到重视、非常有趣而独特的性质：程序变换不会破坏这一表示的正确性。

## 书页 vii

### vii-1

#### Original

The first work was incomplete. It didn't cover the semantics of many common storage classes, and the construction algorithm was not well thought through. But the surprising lesson was that by constructing a uniform representation in which both transformations and analysis could be represented, we could simplify and improve the efficiency of both.

#### 译文

最初的工作并不完整。它没有涵盖许多常见存储类别的语义，构造算法也考虑得不够周全。但一个出人意料的启示是：只要构造一种统一的表示，让变换与分析都能在其中表达，我们就可以同时简化二者并提高二者的效率。

### vii-2

#### Original

Over the next few years, we teamed up with others within IBM to develop not only an efficient method to enter SSA form but also a suite of techniques that each used SSA form and left the representation intact when finished. These techniques included dead code elimination, value numbering, and invariant code motion.

#### 译文

在随后的几年里，我们与 IBM 内部的其他人合作，不仅开发了高效进入 SSA 形式的方法，还开发了一整套使用 SSA 的技术；这些技术在完成处理后仍保持 SSA 表示完好不变。其中包括死代码消除、值编号和循环不变代码外提。

### vii-3

#### Original

The optimizations that we developed are in fact quite simple, but that simplicity is a result of pushing much of the complexity into the SSA representation. The dataflow counterparts to these optimizations are complex, and this difference in complexity was not lost on the community of researchers outside of IBM. Being able to perform optimizations on a mostly functional representation is much easier than performing them on a representation that had all of the warts of a real programming language.

#### 译文

我们开发的优化其实相当简单，但这种简单，是把很大一部分复杂性推入 SSA 表示本身的结果。用数据流方法实现同类优化要复杂得多；IBM 以外的研究群体也清楚地注意到了这种复杂度差异。在一种近似函数式的表示上进行优化，要比在一种带有真实程序设计语言全部“疙瘩”和历史包袱的表示上进行优化容易得多。

### vii-4

#### Original

By today's standards, our original work was not really that useful: there were only a handful of techniques and the SSA form only worked for unaliased variables. But that original work defined the framework for how programming language transformation was to be performed. This book is an expression of how far SSA form has come and how far it needs to go to fully displace the older techniques with more efficient and powerful replacements. We are grateful and humbled that our work has led to such a powerful set of directions by others.

## 译文

以今天的标准衡量，我们最初的工作其实并没有那么实用：当时只有少数几种技术，而且 SSA 形式仅适用于不存在别名关系的变量。但这项早期工作确立了程序设计语言变换应当采用的框架。本书既展示了 SSA 形式迄今取得的进展，也说明它若要用更高效、更强大的替代方案彻底取代旧技术，还需要走多远。我们的工作能够启发后来者开辟出如此强大而多样的研究方向，令我们既感激又深感谦卑。

美国纽约州 Chappaqua / Yorktown Heights，2021 年 3 月

Kenneth Zadeck / Mark N. Wegman

## Foreword 解读

这篇前言给出的并不只是历史回忆，而是一条清晰的技术主线：传统数据流分析把“在每个程序点成立的全部事实”显式求出来；SSA 则通过给每次赋值产生的新值一个独立名字，并在控制流汇合处显式合并值，把关键依赖直接编码进中间表示。于是，很多分析从“稠密地求全局事实”变成“沿相关定义与使用稀疏传播”。

1. \* “静态单赋值”不是运行时只能赋值一次。\* “静态”指编译期中间表示；“单赋值”指每个 SSA 名字在程序文本中只有一个定义。例如源程序中两次写  $x$ ，进入 SSA 后可以写成  $x_1$ 、 $x_2$ 。
2. \* “稀疏”是 SSA 的关键价值。\* 传统稠密分析可能为每个基本块、每个变量维护状态；稀疏分析主要沿定义-使用关系处理真正相关的位置。它不是说程序短了，而是说分析问题的表示减少了无关位置。
3. \*  $\phi$  函数解决控制流汇合。\* 若 if 的两个分支分别定义  $x_1$  和  $x_2$ ，汇合点可写  $x_3 = \phi(x_1, x_2)$ 。 $\phi$  不是普通的运行时函数调用，而是表示“根据实际到达的前驱边选择相应值”的 IR 记号。
4. \* “近似函数式”来自不可变值。\* 每个 SSA 名字只定义一次，之后不再改变，很像函数式语言中的不可变绑定。优化器因此不必反复追问“这里的  $x$  到底是哪次赋值产生的值”。
5. \* 复杂性没有消失，而是被集中管理。\* SSA 上的优化算法之所以简单，是因为重命名、 $\phi$  插入和 SSA 维护承担了复杂性。这是一种工程上的重新分配，而不是免费午餐。
6. \* “只适用于无别名变量”指出了内存的难点。\* 若两个指针可能指向同一内存位置，对其中一个的写入也可能改变另一个读到的值。普通标量 SSA 不能直接把这种关系完全表示出来，因此后来出现了 Memory SSA、 $\psi$ -SSA 等扩展和配套分析。
7. \* “变换不破坏表示正确性”很重要。\* 优化完成后仍保持 SSA 不变量，下一项优化便可以直接复用表示中的依赖信息，不必每一趟都退回原始形式并从头分析。

# Preface | 序言

## 书页 ix

### ix-1

#### Original

This book could have been given several names: “SSA-based compiler design,” or “Engineering a compiler using the SSA form,” or even “The Static Single Assignment form in practice.” But now, if anyone mentions “The SSA book,” then they are certainly referring to this book.<sup>1</sup>

#### 译文

本书本可以采用好几个书名，例如“基于 SSA 的编译器设计”“使用 SSA 形式构建编译器”，甚至“静态单赋值形式实践”。但从今以后，只要有人提到“那本 SSA 书”，指的肯定就是本书。<sup>1</sup>

### ix-2

#### Original

Twelve years were necessary to give birth to a book composed of 24 chapters and written by 31 authors. Twelve years: We are not very proud of this counter performance but here is how it comes: one day, a young researcher (myself) was told by a smart researcher (Jens Palsberg): “we should write a book about all this!” “We” was including Florent Bouchez, Philip Brisk, Sebastian Hack, Fernando Pereira, Jens, and myself. “All this” was referring to all the cool stuff related to register allocation of SSA variables. Indeed, the discovery made independently by the Californian (UCLA), German (U. of Karlsruhe), and French (Inria) research groups (Alain Darte was the one at the origin of the discovery in the French group) was intriguing, as it was questioning the relevance of all the register allocation papers published during the last thirty years. The way those three research groups came to know each other is worth mentioning by itself as without this anecdote the present world would probably be different (maybe not a lot, but most certainly with no SSA book...): A missing pdf file of a CGO article [240] on my web page made Sebastian and Fernando independently contact me. “Just out of curiosity, why are you interested in this paper?” led to a long friendly and fruitful collaboration... The lesson to learn from that story is: do not spend too much time on your web page, it may pay off in unexpected ways!

#### 译文

这本由 31 位作者共同撰写、包含 24 章的书，足足用了十二年才诞生。十二年——我们对这种“反向业绩”可不怎么自豪。事情是这样开始的：有一天，一位年轻研究者（也就是我）听一位聪明的研究者 Jens Palsberg 说：“我们应该把这一切写成一本书！”这里的“我们”包括 Florent Bouchez、Philip Brisk、Sebastian Hack、Fernando Pereira、Jens 和我；“这一切”指的是与 SSA 变量的寄存器分配有关的所有精彩成果。事实上，加州的 UCLA 团队、德国卡尔斯鲁厄大学团队和法国 Inria 团队各自独立得到了一项耐人寻味的发现（法国团队中，这项发现由 Alain Darte 发端）：它对过去三十年发表的大量寄存器分配论文是否仍然重要提出了疑问。这三个研究组相识的经过本身就值得一提；如果没有这段插曲，今天的世界大概会有一点不同——差别也许不大，但几乎肯定不会有这本 SSA 书：因为我个人网页上缺了一篇 CGO 论文 [240] 的 PDF，Sebastian 和 Fernando 分别联系了我。一句“只是好奇，你为什么对这篇论文感兴趣？”开启了一段长久、友好而富有成果的合作……这个故事告诉我们：不要在个人网页上花太多时间，它可能会以意想不到的方式给你回报！

### ix-3 ( 续至 x )

#### Original

But why restricting a book on SSA to just register allocation? SSA was starting to be widely adopted in mainstream compilers, such as LLVM, Hotspot, and GCC, and was motivating many developers and research groups to revisit the different compiler analyses and optimizations for this new form.



## 译文

但是，为什么要把一本讲 SSA 的书局限于寄存器分配呢？当时 SSA 已开始被 LLVM、HotSpot 和 GCC 等主流编译器广泛采用，也促使众多开发者和研究团队针对这种新形式，重新审视各种编译器分析与优化。

## 脚注 1

Original: Put differently, this book fulfils the criterion of referential transparency, which seems to be a minimum requirement for a book about Static Single Assignment form... If this bad joke does not make sense to you, then you definitely need to read the book. If it does make sense, I am pretty sure you can still perfect your knowledge by reading it.

译文：换一种说法，本书满足了“引用透明性”（referential transparency）的判据——对于一本讨论静态单赋值形式的书来说，这似乎是最低要求……如果你没看懂这个冷笑话，那么你肯定需要读这本书；如果你看懂了，我也相当确定，读完本书仍能让你的知识更加完善。

## 书页 x

### x-1

## Original

I was myself lucky to take my first steps in compiler development from 2000 to 2002 in LAO [102], an SSA-based assembly level compiler developed at STMicroelectronics.<sup>2</sup> Thanks to SSA, it took me only four days (including the two days necessary to read the related work) to develop a partial redundancy elimination very similar to the GVN-PRE of Thomas VanDrunen [295] for predicated code! Given my very low expertise, implementing the SSAPRE of Fred Chow (which is not SSA-based—see Chap. 11) would probably have taken me several months of development and debugging, for an algorithm less effective in removing redundancies. In contrast, my pass contained only one bug: I was removing the redundancies of the stack pointer because I did not know what it was used for... There is a lesson to learn from that story: If you do not know what a stack pointer is, do not give up, with substantial efforts you can still expect a position in a compiler group one day.

## 译文

我本人很幸运：2000 至 2002 年间，我在 LAO [102] 中迈出了编译器开发的第一步。LAO 是意法半导体开发的一款基于 SSA 的汇编级编译器。<sup>2</sup> 得益于 SSA，我只用了四天——其中还包括阅读相关工作的两天——就为谓词化代码开发出一种部分冗余消除算法，其做法与 Thomas VanDrunen 的 GVN-PRE [295] 非常相似！考虑到我当时经验极少，如果要实现 Fred Chow 的 SSAPRE（它其实并不基于 SSA，参见第 11 章），我大概需要数月的开发和调试，而且所得算法消除冗余的效果还会更差。相比之下，我写的优化趟只有一个 bug：我把栈指针的“冗余”也消除了，因为我当时不知道它是做什么用的……这个故事的教训是：即使你不知道栈指针是什么，也不要放弃；只要付出足够努力，总有一天你仍有希望在编译器团队里谋得一席之地。

## 脚注 2

Original: More precisely  $\psi$ -SSA—see Chap. 15.

译文：更准确地说，是  $\psi$ -SSA——参见第 15 章。



xi-1

Original

I realized later that many questions were not addressed by any of the existing compiler books: If register allocation can take advantage the structural properties of SSA, what about other low-level optimizations such as post-pass scheduling or instruction selection? Are the extensions for supporting aliasing or predication as powerful as the original form? What are the advantages and disadvantages of SSA form? I believed that writing a book that would address those broader questions should involve the experts of the domain.

译文

后来我意识到，已有的编译器书籍都没有回答许多问题：如果寄存器分配可以利用 SSA 的结构性质，那么后期指令调度、指令选择等其他低层优化又该如何？为支持别名或谓词执行而设计的 SSA 扩展，是否像原始形式一样强大？SSA 形式究竟有哪些优点和缺点？我认为，要写一本回答这些更广泛问题的书，就应当邀请该领域的专家参与。

xi-2 ( 名单续至 xii )

Original

With the help of Sebastian Hack, I decided to organize the SSA seminar. It was held in Autrans near Grenoble in France (yes for those who are old enough, this is where the 1968 winter Olympics were held) and regrouped 55 people (from left to right in the picture—speakers reported in *italic*): Philip Brisk, Christoph Mallon, Sebastian Hack, Benoit Dupont de Dinechin, David Monniaux, Christopher Gautier, Alan Mycroft, Alex Turjan, Dmitri Cheresiz, Michael Beck, Paul Biggar, Daniel Grund, Vivek Sarkar, Verena Beckham, Jose Nelson Amaral, Donald Nguyen, Kenneth Zadeck, James Stanier, Andreas Krall, Dibyendu Das, Ramakrishna Upadrasta, Jens Palsberg, Ondrej Lhotak, Hervé Knochel, Anton Ertl, Cameron Zwarich, Diego Novillo, Vincent Colin de Verdière, Massimiliano Mantione, Albert Cohen, Valerie Bertin, Sebastian Pop, Nicolas Halbwachs, Yves Janin, Boubacar Diouf, Jeremy Singer, Antoniu Pop, Christian Wimmer, Francois de Ferrière, Benoit Boissinot, Markus Schordan, Jens Knoop, Christian Bruel, Florent Bouchez, Laurent Guerby, Benoît Robillard, Alain Darte, Fabrice Rastello, Thomas Heinze, Keshav Pingali, Christophe Guillon, Wolfram Amme, Quentin Colombet, and Julien Le Guen.

译文

在 Sebastian Hack 的帮助下，我决定组织一次 SSA 研讨会。会议在法国格勒诺布尔附近的奥特朗举行——年纪足够大的读者也许记得，1968 年冬季奥运会曾在这里举办——共有 55 人参加。照片中从左到右依次为（演讲者在原书中以斜体标出）：Philip Brisk、Christoph Mallon、Sebastian Hack、Benoit Dupont de Dinechin、David Monniaux、Christopher Gautier、Alan Mycroft、Alex Turjan、Dmitri Cheresiz、Michael Beck、Paul Biggar、Daniel Grund、Vivek Sarkar、Verena Beckham、Jose Nelson Amaral、Donald Nguyen、Kenneth Zadeck、James Stanier、Andreas Krall、Dibyendu Das、Ramakrishna Upadrasta、Jens Palsberg、Ondrej Lhotak、Hervé Knochel、Anton Ertl、Cameron Zwarich、Diego Novillo、Vincent Colin de Verdière、Massimiliano Mantione、Albert Cohen、Valerie Bertin、Sebastian Pop、Nicolas Halbwachs、Yves Janin、Boubacar Diouf、Jeremy Singer、Antoni Pop、Christian Wimmer、Francois de Ferrière、Benoit Boissinot、Markus Schordan、Jens Knoop、Christian Bruel、Florent Bouchez、Laurent Guerby、Benoît Robillard、Alain Darte、Fabrice Rastello、Thomas Heinze、Keshav Pingali、Christophe Guillon、Wolfram Amme、Quentin Colombet 和 Julien Le Guen。

### xii-1

#### Original

The goal of this seminar was twofold: first, have some of the best-known compiler experts climb to the top of the local mountain peak; second, regroup potential authors of the book, and work together on finding who could contribute to the writing. At the end of the week, some of the participants were still wondering why they accepted this invitation from this crazy mountain guy (although everyone made it to the top—I have pictures!), but the main layout of the book was roughly set up with authors for each chapter. The agreed objective was to have a book similar in spirit to a textbook: one that would cover most of the phases of an optimizing compiler; one that would use consistent notations and terminology across all the chapters of the book; one that would have all the required notions defined (possibly in a different chapter) before they are used. But it was not meant to be literally a textbook as it was for advanced compiler developers only. For a good textbook, we suggest, for example, the Tiger book [10] by Andrew Appel instead. In this current book, some of the chapters were constrained by a page limit and might be really tough to understand otherwise. Paul Biggar created a LaTeX infrastructure, and I started pressing authors to fulfil their duty as soon as possible, convinced that within a year the book would be available... Lesson learned from that seminar: if a mountain climber asks you to join him for a “short” hike, tell him you have to work on the writing of a book.

#### 译文

这次研讨会会有两个目标：第一，让几位最知名的编译器专家爬上当地山峰；第二，把本书的潜在作者召集起来，共同确定谁可以参与写作。一周结束时，一些参会者还在琢磨，自己为什么会接受这个“登山疯子”的邀请——不过所有人都成功登顶了，我有照片为证！与此同时，本书的总体框架也大致确定，各章都有了作者。大家商定，要让这本书在精神上类似教材：覆盖优化编译器的大多数阶段；全书各章使用一致的符号和术语；所有必需概念都应在使用前定义好，定义也可以放在其他章节。不过，它并不打算成为严格意义上的教材，因为目标读者仅限高级编译器开发者。若要选择一本优秀教材，我们推荐 Andrew Appel 的“虎书”[10]。本书有些章节受页数限制，否则可能真的会难以理解。Paul Biggar 搭建了 LaTeX 基础设施，我也开始催促作者尽快履行职责，并确信书将在一年内问世……这次研讨会留下的教训是：如果一个登山者邀请你参加一次“短途”徒步，就告诉他你得去写书。

### xii-2 ( 续至 xiii )

#### Original

This is when the smart guy declined to participate in the book adventure, and I suspect his intuition was telling him that the project was a bit too ambitious... It indeed turned out to be a very difficult task, especially for a young researcher. Writing a book by myself would probably have been much easier, but also less interesting. Concatenating independent chapters as one would have done for a compilation of the state of the art would also have been infinitely easier but would have lacked coherence and brought little or no added value. So, each chapter required significant rewriting from the authors compared to the original publishings, and the main reason it took such a long time is that it turns out to be extremely difficult to impose deadlines when there are no strong constraints that motivate those deadlines, as opposed for instance to the pressure to submit conference articles. We thought initially that six months would be enough for the different authors to write their chapter, and I anticipated two full months without any critical duty right after that period so as to be able to orchestrate the reviewing process: I wanted to review all chapters myself, but also have the different authors to do cross-reviewing of other chapters. After the deadline, it was already obvious that there was no way we would have a book within the time limit initially expected: most of the chapters were missing, and some were a carbon copy from the corresponding paper... Despite this, the reviewing process started, and it was clear a very deep reviewing was required.

## 译文

就在这时，那位聪明人退出了这场写书冒险；我怀疑他的直觉已经告诉他，这个项目有些过于雄心勃勃……。后来的事实证明，这的确是一项极其艰巨的任务，对一名年轻研究者尤其如此。独自写书大概会容易许多，但也会无趣许多。若像编纂前沿论文集那样，把彼此独立的章节直接拼在一起，当然更是轻而易举；可那样会缺乏连贯性，也几乎不会带来附加价值。因此，与作者原先发表的论文相比，每一章都需要大幅重写。全书耗时如此之久，主要是因为：如果没有强力约束来支撑期限，给人设定截止日期竟然极其困难；这与提交会议论文时的压力截然不同。我们最初以为，不同作者用六个月就足以写完各自章节；我还预留了紧随其后的整整两个月，期间不安排任何关键任务，以便统筹评审。我打算亲自审阅所有章节，同时让不同作者交叉评审彼此的章节。截止日期过后，显然不可能在原定时限内完成本书：大多数章节尚未交稿，有些则只是对应论文的原样复制……。尽管如此，评审还是启动了，而且很快就能看出，必须进行极其深入的审阅。

## 书页 xiii

### xiii-1

#### Original

Indeed, almost every chapter contained at least one important technical flaw; people have different definitions (with slight but still important differences) for the same notion; some spent pages developing a new formalism that is already covered by existing well-established ones. But a deep review of a single chapter roughly takes a week... Starts a long walk in a tunnel where you context switch from one chapter review to another, try to orchestrate the cross-reviews, ask for the unfinished chapters to be completed, pray that the missing chapters finally change their status to "work-in-progress," and harass colleagues with metre-long back and forth email threads trying to finally clarify this one 8-word sentence that may be interpreted in the wrong way. You then realize no one will be able to implement the agreed changes in time because they got themselves swamped by other duties (the ones from their real job), and when these are available that is when you get yourself swamped by other duties (the ones from your real job), and your 2-month window is already flying away, leaving yourself realizing that the next one will be next year. And time flies because, for everybody including yourself, there is always something else more urgent than the book itself, and messages in this era of near-instantaneous communication behave as if they were using strange paths around the solar system, experiencing the joys of time dilation with 6-month return trips around Saturn's rings. More prosaically, I will lay the blame on the incompatibility between the highly imposed constraints and the absence of a strong common deadline for everybody. And so, the final lesson to learn from that experience is that if a mountain climber proposes to join him for a "short" hike, go for it: forget about the book, it will be less painful!

## 译文

事实上，几乎每一章都至少包含一个重要的技术缺陷；不同的人会对同一概念给出不同定义，差异虽小却依然重要；还有人花费数页篇幅发展一种新形式体系，殊不知成熟的既有体系早已覆盖了同样内容。然而，深入审阅一章大约就要花上一周……。于是，一场漫长的隧道跋涉开始了：你不断在不同章节的评审之间切换上下文，努力协调交叉评审，催促尚未完成的章节早日完稿，祈祷那些仍然缺失的章节终于把状态改成“进行中”；你还用长达一米、来回往复的邮件串“骚扰”同事，只为最终说清一个仅有 8 个词、却可能遭到误解的句子。接着你发现，谁也无法及时落实已经商定的修改，因为他们都被其他职责——来自本职工作的职责——淹没了；等他们终于有空时，恰好又轮到你被自己的其他职责——同样来自本职工作的职责——淹没。你预留的两个月窗口早已飞逝，下一个这样的窗口要等到来年。时间不断流逝，因为对每个人——包括你自己——来说，总有别的事情比这本书更紧急。在这个通信近乎即时的时代，消息却仿佛选择了绕行太阳系的诡异路径，享受着时间膨胀的乐趣，绕土星环往返一趟就要六个月。说得朴素一些，我认为问题在于：一方面项目受到许多严格约束，另一方面所有人却没有一个强有力的共同截止日期，二者互不相容。因此，这段经历带来的最后一条教训是：如果一个登山者邀请你参加一次“短途”徒步，那就去吧；忘掉写书这事儿，徒步反而没那么痛苦！

### Original

A few years after we initiated this project, as I was depressed by the time all this was taking, I met Charles Glaser from Springer who told me: “You have plenty of time: I have a book that took 15 years to be completed.” At the time I am writing those lines, I still have a long list of to-dos, but I do not want to be the person Charles mentions as “Do not worry, you cannot do worse than this book they started writing 15 years ago and which is still under construction...” Sure, you might still find some typos or inconsistencies, but there should not be many: the book regroups the knowledge of many talented compiler experts, including substantial unpublished materials, and I am proud to release it today.

### 译文

项目启动几年后，我因它耗时之久而倍感沮丧。那时我遇到 Springer 的 Charles Glaser，他对我说：“你的时间还多得很：我手上有一本书，花了 15 年才完成。”写下这些文字时，我的待办事项清单依然很长，但我可不想成为 Charles 口中的那个人，让他说：“别担心，你不可能比这本书更糟——他们 15 年前就开始写了，现在还没写完……”当然，你或许仍会在书中发现一些错别字或不一致之处，但应该不会太多。本书汇聚了众多优秀编译器专家的知识，其中还包括大量未发表的材料；今天能够将它正式出版，我深感自豪。

法国格勒诺布尔，2021 年 2 月  
Fabrice Rastello

## Preface 解读

1. \*“引用透明”笑话。\* 在函数式语义中，若一个表达式可被它的值替换而不改变程序行为，就称其具有引用透明性。作者说“The SSA book”可以直接指代本书，故意把日常语言中的“指代”偷换成技术术语；SSA 的不可变名字又与函数式、引用透明的直觉相呼应。
2. \*SSA 与寄存器分配。\* SSA 直接暴露值的定义-使用关系和活跃范围结构，使某些干涉图具有特殊性质，能让合并、着色或销毁 SSA 的过程得到更强的结构保证。这也是作者最初打算写书的技术起点。
3. \*PRE、GVN-PRE 与 SSAPRE。\* 部分冗余消除 ( Partial Redundancy Elimination , PRE ) 消除“只在某些路径上已经算过”的重复表达式；GVN-PRE 把全局值编号与 PRE 结合。文中所谓 SSAPRE 是 Fred Chow 等人的经典算法名称；作者特意提醒，它虽然名为 SSAPRE，其核心表示并不是本书语境下的 SSA，故而形成一个容易误解的命名反差。
4. \*谓词化代码与  $\psi$ -SSA。\* 谓词执行用条件谓词控制指令是否生效，从而减少显式分支。普通  $\phi$  表示控制流汇合； $\psi$ -SSA 则用于表达谓词化定义之间的合并及优先关系，适合汇编级、谓词化的中间表示。
5. \*栈指针为什么不能随便“消除”。\* 栈指针记录当前栈帧位置，函数调用、局部变量、保存寄存器和返回地址等都可能依赖它。若优化器只看到算术等价，却没有正确建模这些隐含语义，就可能把必要的栈指针更新当成冗余代码删除。
6. \*低层优化为何是检验 SSA 的试金石。\* 指令选择、调度、寄存器分配会面对机器约束、别名、异常、谓词执行和隐式寄存器效果。SSA 在标量高层优化中很优雅，但进入机器相关阶段后必须扩展，书名中的“SSA-based compiler design”正是要讨论这整条工程链，而不只是经典 SSA 构造。
7. \*这不是入门教材。作者强调本书面向高级编译器开发者，目标是让 31 位专家的章节在符号、术语和前置知识上保持一致。所谓“虎书”通常指 Andrew W. Appel 的 Modern Compiler Implementation\* 系列，更适合作为系统入门教材。
8. \*两篇文字的分工。\* Foreword 由 SSA 奠基者回顾“为什么需要 SSA、它改变了什么”；Preface 由主编讲述“为什么需要这本书、它为何用了十二年”。前者提供技术史与核心思想，后者交代本书的范围、受众和协作方式。

## 术语速查

| English                              | 中文        | 简要含义                             |
|--------------------------------------|-----------|----------------------------------|
| Static Single Assignment (SSA)       | 静态单赋值形式   | 每个 SSA 名字只有一个静态定义的中间表示性质         |
| intermediate representation (IR)     | 中间表示      | 编译器内部承载程序语义并接受分析、优化的表示           |
| dataflow analysis                    | 数据流分析     | 在控制流图上求解事实如何产生、传播与汇合             |
| sparse representation                | 稀疏表示      | 只在与事实相关的位置显式承载或传播信息              |
| def-use chain                        | 定义-使用链    | 从值的定义连接到使用该值的位置                  |
| birthpoint                           | 出生点       | 某个值或合并值需要被引入的位置；是 $\phi$ 插入思想的前身 |
| phi-function / $\phi$ -function      | $\phi$ 函数 | 在控制流汇合点按来路选择不同 SSA 值的伪操作         |
| constant propagation                 | 常量传播      | 证明值为常量并用常量替换使用处                  |
| dead code elimination                | 死代码消除     | 删除不会影响可观察结果的代码                   |
| value numbering                      | 值编号       | 识别计算结果相同的表达式或值                   |
| invariant code motion                | 循环不变代码外提  | 把循环中结果不变的计算移出循环                  |
| aliasing                             | 别名        | 两个名字或指针可能引用同一存储位置                |
| register allocation                  | 寄存器分配     | 把程序值映射到有限机器寄存器，必要时溢出到内存          |
| predication                          | 谓词执行      | 用条件谓词控制指令生效，减少显式控制流分支            |
| partial redundancy elimination (PRE) | 部分冗余消除    | 消除只在部分到达路径上重复的计算                 |
| stack pointer                        | 栈指针       | 指向当前运行时栈位置的机器寄存器或抽象状态            |

## 一句话总结

SSA 的根本贡献不是“给变量加下标”这么简单，而是把程序中值的来源、使用和控制流汇合显式编码进一种可持续维护的稀疏表示，从而让一连串编译器分析与优化能够共享结构、降低复杂度；本书则试图把这种思想从经典标量优化扩展到完整的现代优化编译器工程。